



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Y
UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

Guía de Aprendizaje y Manual de Prácticas
Diseño Digital con SystemVerilog y FPGA Gowin Tang Nano 9K

Alumno:

Altamira Cisneros Luis Gerardo

Profesor:

Dr. Mauricio Alonso Sánchez

5 de junio de 2026

Índice

1. Introducción al Entorno de Trabajo	3
2. Conceptos Fundamentales	4
2.1. ¿Qué es un FPGA?	4
2.2. ¿Qué es SystemVerilog?	4
2.3. La Tarjeta Tang Nano 9K	4
2.4. Repositorio Oficial del Proyecto	4
3. Preparación del Entorno de Desarrollo	5
3.1. Paso 1: Instalación de Dependencias del Sistema	5
3.2. Paso 2: Instalación de Gowin IDE (EDA)	6
3.3. Paso 3: Instalación de Visual Studio Code	7
3.4. Paso 4: Extensiones de VS Code para SystemVerilog	8
3.5. Paso 5: Clonar el Repositorio del Proyecto	9
4. Fundamentos de SystemVerilog para FPGA	10
4.1. Módulo Top y Puertos	10
4.2. Asignaciones Combinacionales (assign)	10
4.3. Lógica Secuencial (always_ff)	11
4.4. Estructuras de Control	11
4.5. Tipos de Datos	11
4.6. Operadores	12
5. Guía de Uso y Ejecución de las Prácticas	13
5.1. Paso 1: Seleccionar la Placa FPGA (Solo la primera vez)	13
5.2. Paso 2: Ejecutar una Práctica	15
5.3. Otros Scripts Útiles	16
5.4. Resumen del Flujo de Trabajo Diario	17
6. Solución de Problemas Comunes	17
7. Prácticas de Laboratorio	18
7.1. Práctica 1: Encendido de un LED Estático	18
7.2. Práctica 2: El Botón que enciende el LED	19
7.3. Práctica 3: Compuertas Lógicas Básicas	20
7.4. Práctica 4: El LED que Parpadea	21
7.5. Práctica 5: Contador Binario de 8 Bits	22
7.6. Práctica 6: Display de 7 Segmentos (Dígito Simple)	23
7.7. Práctica 7: Multiplexación de Display y Antirrebote	24
7.8. Práctica 8: Semáforo con LEDs (Máquina de Estados)	25

7.9. Práctica 9: Introducción a Gráficos LCD (Coordenadas y Colores)	26
7.10. Práctica 10: Movimiento Automático en Pantalla LCD	27
7.11. Práctica 11: Control Interactivo por Botones	28
7.12. Práctica 12: Pelota Rebotante (Animación Física LCD)	29
7.13. Práctica 13: Semáforo Animado en LCD	30
7.14. Práctica 14: Radar y Medición de Distancia Ultrasónica	31
8. Extendiendo el Proyecto: Agregando Periféricos	32
8.1. Agregar un Módulo Personalizado	32
8.2. Mapeo de Pines	32
8.3. Síntesis Automatizada	32
8.4. Recomendaciones	33
9. Recomendaciones de Diseño y Buenas Prácticas	34
10. Conclusiones	34
11. Anexo de Ecuaciones	35

1. Introducción al Entorno de Trabajo

El diseño de sistemas digitales ha evolucionado de la conexión física de compuertas lógicas al uso de lenguajes de descripción de hardware (HDLs) implementados sobre arreglos de compuertas programables en campo (**FPGAs**).

Esta guía de aprendizaje utiliza la tarjeta de desarrollo **Gowin Tang Nano 9K**, basada en el chip de bajo costo GW1NR-9C. Para interactuar con el entorno exterior, la placa se conecta a un módulo de expansión equipado con un circuito integrado **TM1638** (que maneja 8 displays de 7 segmentos, 8 botones de entrada y 8 LEDs indicadores) y una pantalla **LCD de 480x272 píxeles**.

Todo el material de esta guía está basado en el trabajo de **Yuri Panchul** y su repositorio basics-graphics-music, adaptado específicamente para la placa Tang Nano 9K y su módulo de expansión.

El flujo de diseño consta de:

1. **Modelado:** Escritura de código SystemVerilog (‘.sv’).
2. **Síntesis y P&R:** Traducción del código a una red de compuertas físicas usando el software **Gowin IDE**.
3. **Programación:** Carga del archivo binario (‘.fs’) a la SRAM o memoria Flash del FPGA mediante JTAG.

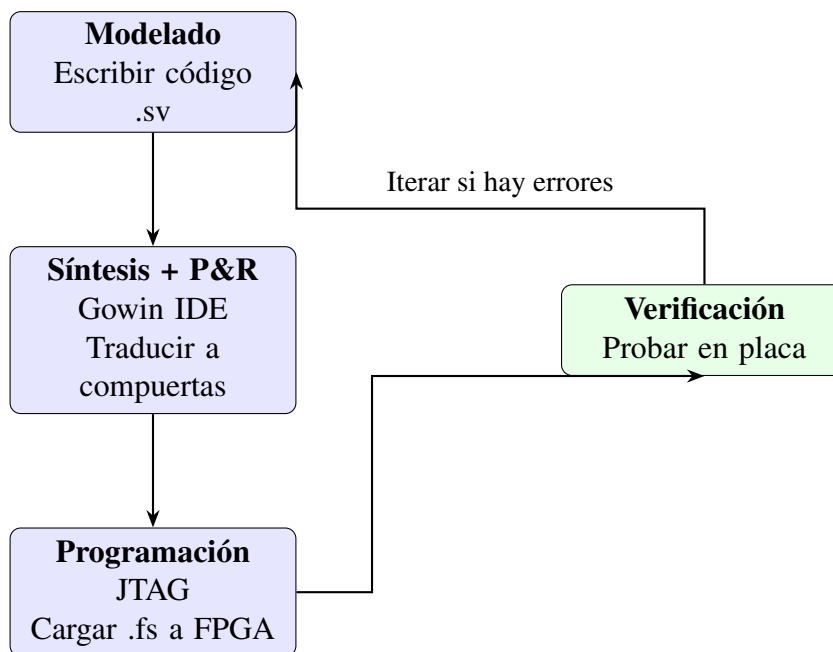


Figura 1: Flujo de diseño digital en FPGA.

2. Conceptos Fundamentales

2.1. ¿Qué es un FPGA?

Un **Field-Programmable Gate Array (FPGA)** es un circuito integrado que contiene una matriz de bloques lógicos configurables, interconectados mediante una red de rutas programables. A diferencia de un microcontrolador que ejecuta instrucciones secuenciales, un FPGA implementa **circuitos digitales físicos** descritos en un lenguaje de hardware. Al programarlo se define la función lógica, las conexiones y la arquitectura del sistema, logrando un paralelismo real y un rendimiento determinístico.

2.2. ¿Qué es SystemVerilog?

SystemVerilog es un lenguaje de descripción de hardware (HDL) que extiende al clásico Verilog con capacidades avanzadas de modelado, verificación y síntesis. En este manual se utiliza para describir el comportamiento de circuitos combinacionales y secuenciales. Las herramientas de síntesis traducen este código a la configuración física del FPGA.

2.3. La Tarjeta Tang Nano 9K

La **Gowin Tang Nano 9K** es una placa de desarrollo compacta basada en el FPGA GW1NR-9C, que incluye:

- 8640 LUTs (look-up tables), suficiente para diseños de mediana complejidad.
- Oscilador de 27 MHz integrado.
- Conector para LCD de 480x272 píxeles.
- Pines GPIO para módulos externos (sensores, botones, etc.).
- Programación vía interfaz JTAG con cable USB incluido.

Su bajo costo y la disponibilidad del entorno de desarrollo gratuito **Gowin IDE** la hacen ideal para la enseñanza y prototipado rápido.

2.4. Repositorio Oficial del Proyecto

Todo el código fuente, scripts y ejemplos de este manual se encuentran disponibles en GitHub:

https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K

Este repositorio es una adaptación del trabajo original de **Yuri Panchul** (<https://github.com/yuri-panchul/basics-graphics-music>), modificado y optimizado para la placa Gowin Tang Nano 9K con

módulo de expansión LCD y TM1638.

3. Preparación del Entorno de Desarrollo

A continuación se detallan todos los pasos necesarios para configurar el entorno de desarrollo en **Ubuntu Linux** (22.04 LTS o superior). Se asume que el lector tiene acceso a una terminal y conexión a internet.

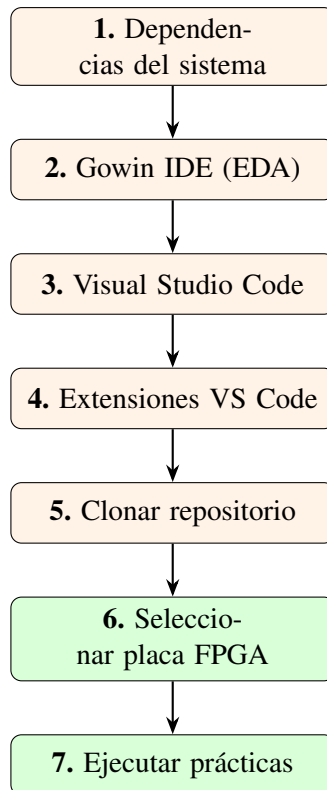


Figura 2: Resumen del proceso de instalación y configuración (7 pasos).

3.1. Paso 1: Instalación de Dependencias del Sistema

Abrir una terminal (Ctrl+Alt+T) y ejecutar los siguientes comandos para instalar las herramientas base:

Terminal

```
$ sudo apt update  
$ sudo apt install git openfpgaloader
```

Explicación de cada paquete:

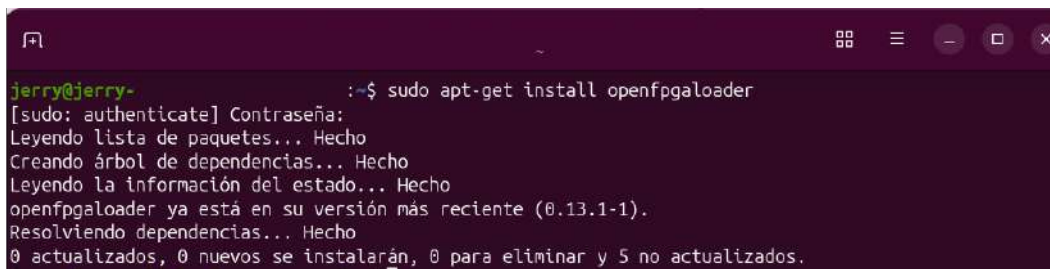
- **git:** Sistema de control de versiones para clonar el repositorio.
- **openfpgaloader:** Herramienta de código abierto para programar FPGAs de diversas marcas (incluyendo Gowin) desde Linux.

Nota sobre permisos USB (reglas udev)

Para poder programar la placa FPGA sin permisos de superusuario (sudo), el sistema Linux requiere reglas de udev. Si experimenta errores al detectar la placa, agregue su usuario al grupo de acceso USB y recargue las reglas:

```
$ sudo usermod -aG plugdev $USER  
$ sudo udevadm control --reload-rules && sudo udevadm trigger
```

Posteriormente, desconecte y vuelva a conectar el cable USB de la placa FPGA.



```
jerry@jerry- ~$ sudo apt-get install openfpgaloader  
[sudo: authenticate] Contraseña:  
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias... Hecho  
Leyendo la información del estado... Hecho  
openfpgaloader ya está en su versión más reciente (0.13.1-1).  
Resolviendo dependencias... Hecho  
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 5 no actualizados.
```

Figura 3: Instalación de Open FPGA Loader.

3.2. Paso 2: Instalación de Gowin IDE (EDA)

El software oficial de Gowin es necesario para sintetizar los diseños y generar el archivo binario para la FPGA.

1. **Crear una cuenta en Gowin:** Visitar https://www.gowinsemi.com/en/support/download_eda/ y registrarse de forma gratuita.
2. **Descargar Gowin EDA:** Acceder a la sección de descargas y seleccionar la versión educativa para Linux: **Gowin V1.9 Education (Linux)**. Descargar el archivo `.tar.gz`.

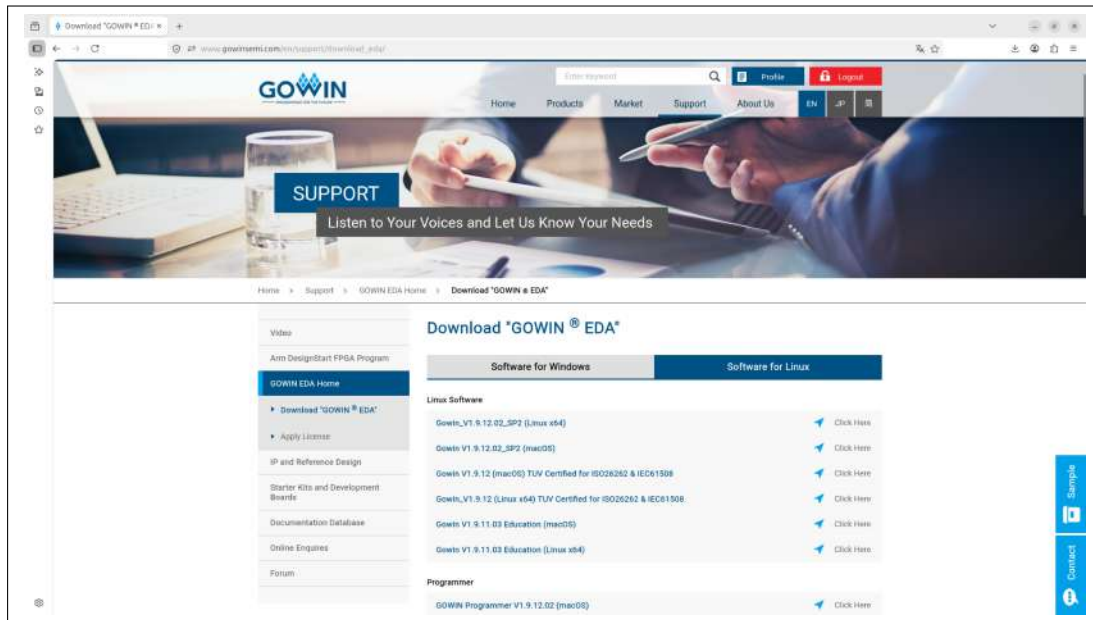


Figura 4: Página de descarga de Gowin EDA.

3. **Extraer el archivo:** En la terminal, navegar a la carpeta de descargas y ejecutar:

Terminal

```
mkdir -p ~/gowin  
tar -xf Gowin_V1.9.XX.XX_Education_Linux.tar.gz  
-C ~/gowin --strip-components=1
```

4. **Verificar la instalación:** Los scripts del proyecto buscarán automáticamente Gowin EDA en `~/gowin` o `/opt/gowin`.

3.3. Paso 3: Instalación de Visual Studio Code

Aunque Gowin IDE incluye un editor propio, se recomienda utilizar **Visual Studio Code** por su ligereza, extensiones y terminal integrada.

Terminal

```
$ sudo snap install code --classic
```

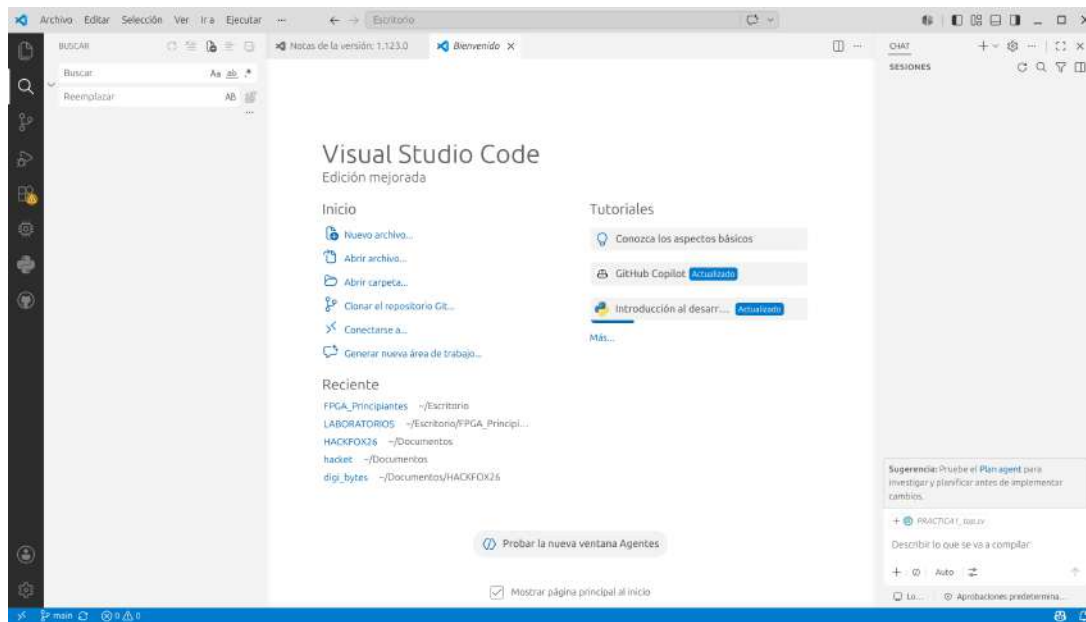


Figura 5: Pantalla de bienvenida de Visual Studio Code.

3.4. Paso 4: Extensiones de VS Code para SystemVerilog

Para tener resaltado de sintaxis, autocompletado y navegación de código, instalar las siguientes extensiones:

1. Abrir VS Code.
2. Ir a la sección de extensiones (Ctrl+Shift+X).
3. Buscar e instalar:
 - **Verilog-HDL/SystemVerilog** (por mshr-h): Proporciona coloreado de sintaxis, autocompletado e integración con linters para Verilog y SystemVerilog.
 - **Verilog/SystemVerilog Tools** (por eirikpre): Ayuda con la navegación de código, instancia-
ción de módulos y utilidades de productividad.

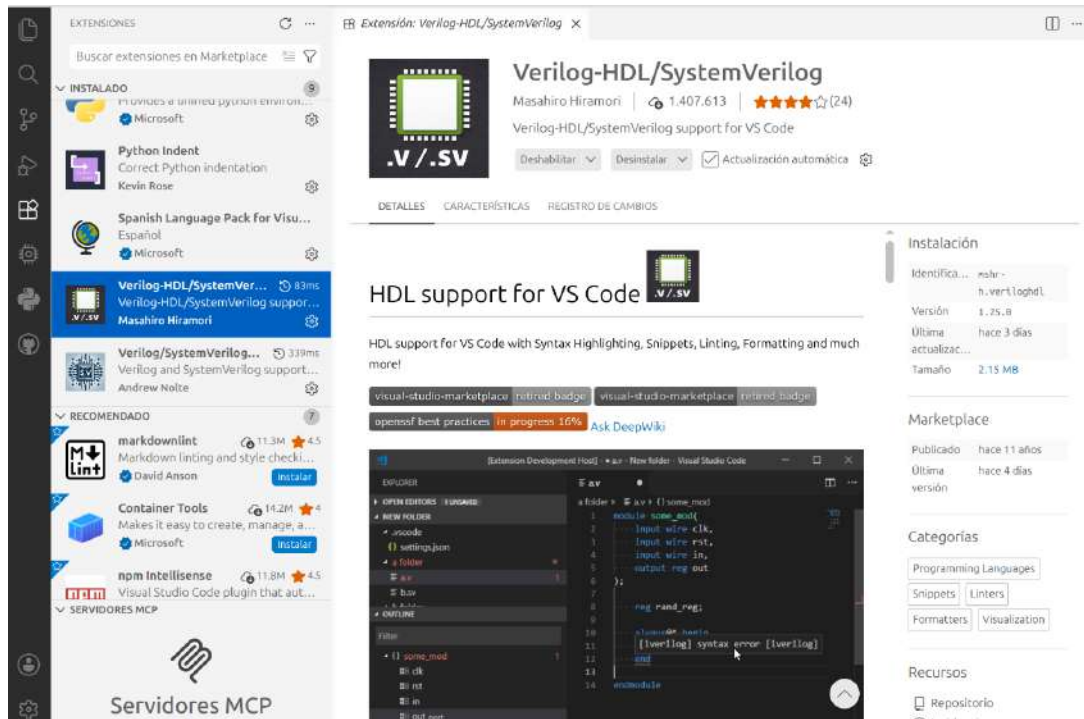


Figura 6: Extensiones recomendadas para SystemVerilog en VS Code.

3.5. Paso 5: Clonar el Repositorio del Proyecto

Para obtener todos los archivos de las prácticas:

1. Abrir una terminal en la carpeta donde se desea descargar el proyecto.
2. Ejecutar:

Terminal

```
git clone https://github.com/JERRYALTAMIRA/
FPGA_PRINCIPIANTES_TANG_NANO_9K.git
```

3. Entrar al directorio descargado:

Terminal

```
cd FPGA_PRINCIPIANTES_TANG_NANO_9K
```

4. Para actualizar el repositorio en el futuro:

Terminal

```
git pull
```

4. Fundamentos de SystemVerilog para FPGA

A continuación se resumen los conceptos básicos que se aplican en todas las prácticas. Se recomienda leerlos antes de comenzar.

4.1. Módulo Top y Puertos

En SystemVerilog, un diseño se estructura en **módulos**. El módulo de nivel más alto (top) contiene los puertos que se conectan físicamente a los pines del FPGA. Cada puerto se declara con una dirección:

- `input logic` – señal que entra al módulo (ej. botón, reloj).
- `output logic` – señal que sale del módulo (ej. LED, segmento).
- `inout logic` – señal bidireccional (usada con cuidado).

Ejemplo mínimo:

```
1  module practicas_top (  
2      input  logic clock,  
3      input  logic reset,  
4      input  logic [7:0] key,  
5      output logic [7:0] led  
6  );  
7      // descripción del circuito  
8      endmodule  
9
```

4.2. Asignaciones Combinacionales (assign)

La instrucción `assign` crea una conexión continua, equivalente a un cableado físico. Se usa para lógica combinatorial.

```
1      assign led[0] = ~key[0];    // LED encendido mientras se presiona el  
2      botón
```

4.3. Lógica Secuencial (always_ff)

Para registrar valores (flip-flops) se emplea el bloque `always_ff` con la sensibilidad al flanco de subida del reloj y a una señal de reset asíncrona.

```
1      logic [7:0] counter;  
2      always_ff @(posedge clock or posedge reset) begin  
3          if (reset)          counter <= 8'h00;  
4          else                counter <= counter + 1;  
5      end  
6
```

La asignación dentro de `always_ff` debe ser no bloqueante (`<=`) para asegurar la correcta inferencia de registros.

4.4. Estructuras de Control

Dentro de los bloques `always` se pueden utilizar las mismas estructuras de los lenguajes de programación:

- `if-else` para condiciones.
- `case` para selección múltiple (muy usado en decodificadores).
- `for` para bucles de generación (solo dentro de bloques `generate` o para lógica repetitiva que se resuelve en tiempo de síntesis).

Ejemplo con `case`:

```
1      case (estado)  
2          2'b00: led = 8'b00000001;  
3          2'b01: led = 8'b00000010;  
4          default: led = 8'b00000000;  
5      endcase  
6
```

4.5. Tipos de Datos

SystemVerilog introduce el tipo `logic` que reemplaza a los tradicionales `wire` y `reg`. Para buses se utiliza la notación `[N:0]` (índices alto: bajo). Ejemplo: `logic [7:0] bus_dato;` (8 bits).

4.6. Operadores

- Operadores bit a bit: `&`, `|`, `^`, `~`
- Operadores lógicos: `&&`, `||`, `!`
- Concatenación: `{a, b}` une señales.
- Aritméticos: `+`, `-`, `*` (sintetizable si el tamaño es limitado).

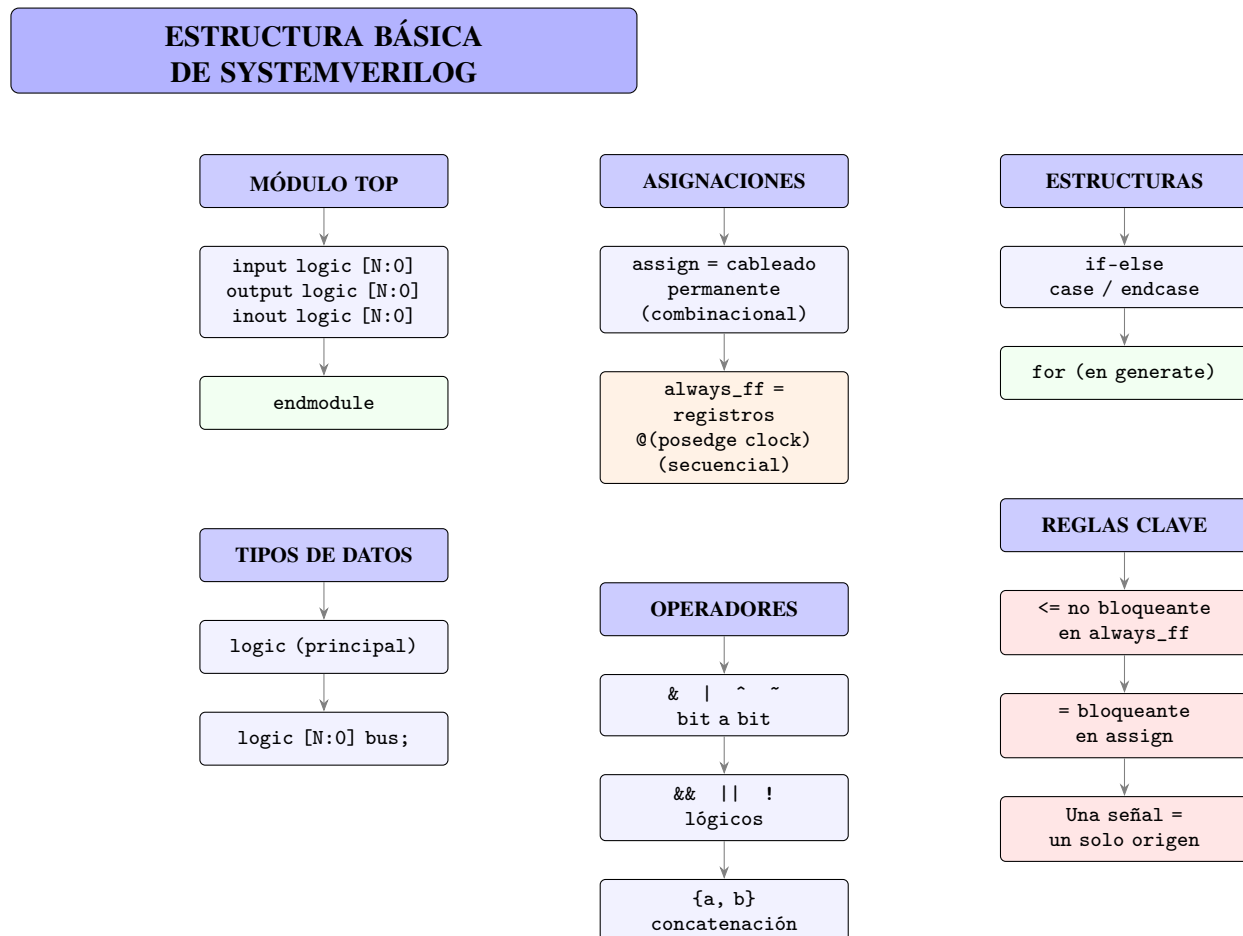


Figura 7: Resumen visual de los fundamentos de SystemVerilog para FPGA.

Con estos fundamentos se puede entender el código de las prácticas siguientes. Se recomienda consultar la documentación completa de SystemVerilog para profundizar.

5. Guía de Uso y Ejecución de las Prácticas

Una vez instalado todo el entorno, se puede proceder a compilar y cargar los diseños en la FPGA.

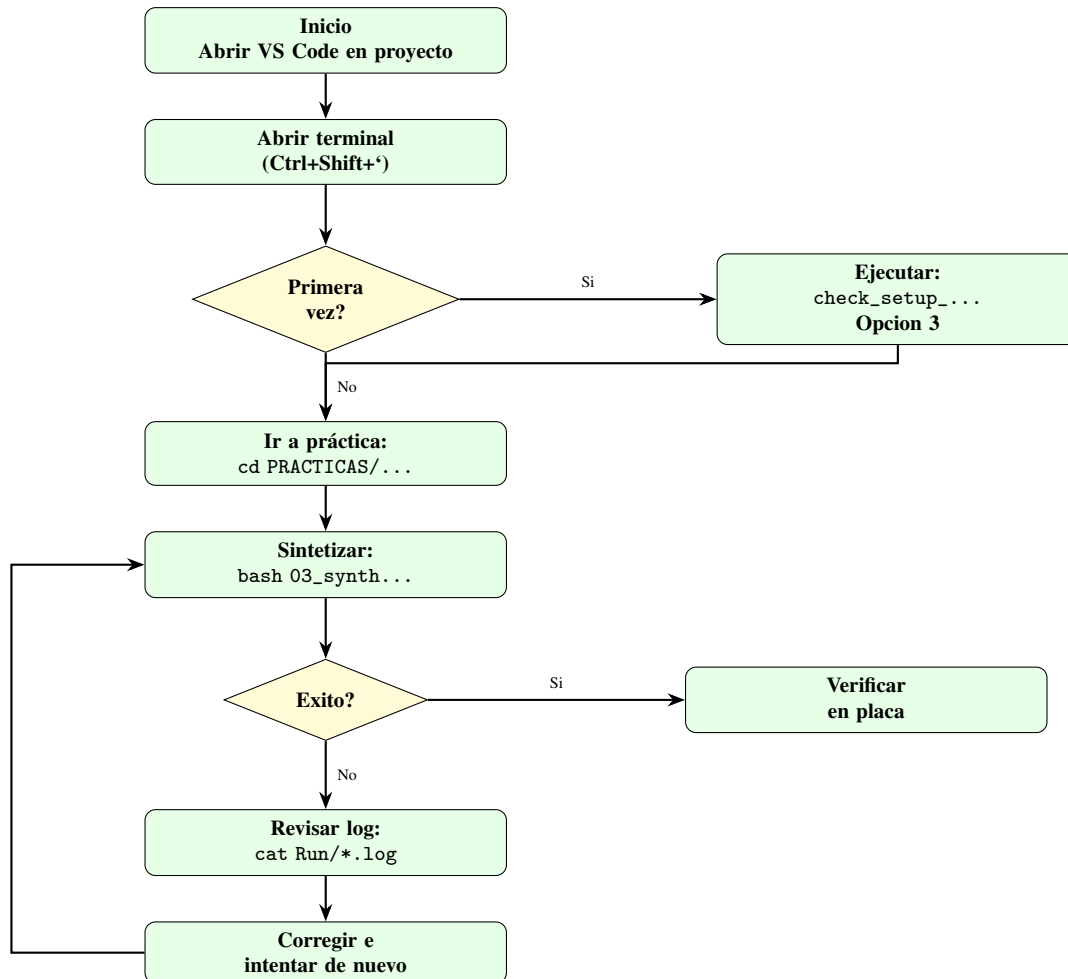


Figura 8: Diagrama de flujo para ejecutar una práctica.

5.1. Paso 1: Seleccionar la Placa FPGA (Solo la primera vez)

Antes de ejecutar cualquier práctica, se debe indicar al sistema qué placa se está utilizando.

1. Abrir el proyecto en VS Code: File > Open Folder y seleccionar la carpeta FPGA_PRINCIPIANTES_TANG_NANO_9K.
2. Abrir la terminal integrada de VS Code: Terminal > New Terminal o con el atajo Ctrl + Shift + '.

3. En la terminal integrada (asegurarse de estar en la raíz del proyecto), ejecutar:

```
bash check_setup_and_choose_fpga_board.bash
```



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
jerry@jerry-ThinkPad-X1-Yoga-3rd:~/Escritorio/FPGA_Principiantes$ bash check_setup_and_choose_fpga_board.bash

check_setup_and_choose_fpga_board.bash: The currently selected FPGA board: tang_nano_9k_lcd_480_272_tm1638_practicas. Please select an FPGA board among the following supported:
1) tang_nano_9k_lcd_480_272_tm1638          3) tang_nano_9k_lcd_480_272_tm1638_practicas
2) tang_nano_9k_lcd_480_272_tm1638_hackathon  4) exit
Your choice (a number): 3
```

Figura 9: Inicio de la ejecución del script de selección de placa.

4. Aparecerá una lista de placas soportadas. Buscar la opción que dice exactamente (sin el símbolo # al inicio):

```
tang_nano_9k_lcd_480_272_tm1638_practicas
```



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
jerry@jerry-ThinkPad-X1-Yoga-3rd:~/Escritorio/FPGA_Principiantes$ bash check_setup_and_choose_fpga_board.bash

check_setup_and_choose_fpga_board.bash: FPGA board selected: tang_nano_9k_lcd_480_272_tm1638_practicas

check_setup_and_choose_fpga_board.bash: Created an FPGA board selection file: "/home/jerry/Escritorio/FPGA_Principiantes/fpga_board_selection"
Would you like to create the new run directories for the synthesis of all labs in the package, based on your FPGA board selection? We recommend to do this if you plan to work with synthesis GUI rather than with the synthesis scripts. [y/n] y
```

Figura 10: Selección de la placa Tang Nano 9K en el menú interactivo.

5. Escribir el número correspondiente (generalmente 3) y presionar Enter.
6. Cuando pregunte si se desea usar la GUI de Gowin, seleccionar N (No) para continuar con el flujo por terminal.

Nota importante: Las líneas que comienzan con # son comentarios y están deshabilitadas. Solo se debe seleccionar la línea que NO tiene # al inicio.

5.2. Paso 2: Ejecutar una Práctica

Cada práctica se encuentra en su propia carpeta dentro del directorio PRACTICAS/. Para compilar y cargar cualquier práctica:

1. En la terminal integrada de VS Code, navegar a la práctica deseada. Por ejemplo, para la Práctica 1:

```
cd PRACTICAS/PRACTICA1
```

2. Ejecutar el script de síntesis y programación:

```
bash 03_synthesize_for_fpga.bash
```

3. El script realizará automáticamente:

- a) **Síntesis lógica:** Convierte el código SystemVerilog en una red de compuertas.
- b) **Place & Route:** Asigna las compuertas a ubicaciones físicas del chip.
- c) **Generación de bitstream:** Crea el archivo .fs en la carpeta Run/.
- d) **Programación:** Carga el bitstream en la FPGA vía JTAG.

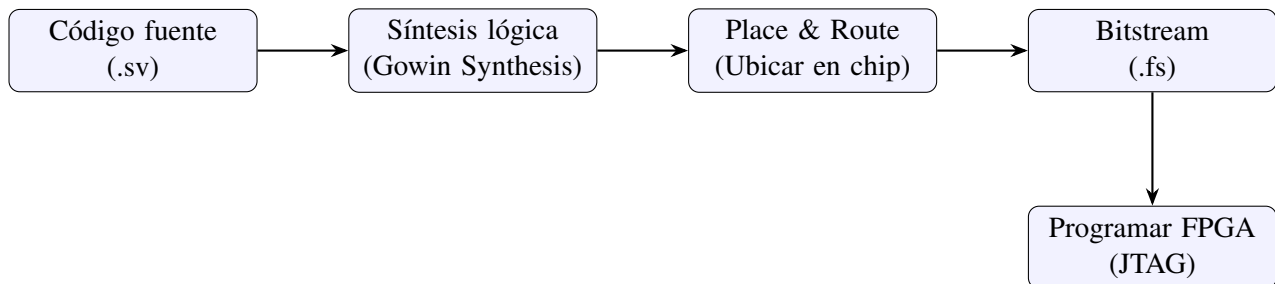


Figura 11: Proceso interno del script 03_synthesize_for_fpga.bash.

4. Si la síntesis es exitosa, se mostrará un mensaje similar a:



```

jerry@jerry-ThinkPad-X1-Yoga-3rd:~/Escritorio/FPGA_Principiantes/PRACTICAS/PRAC
TICA1$ bash 03_synthesize_for_fpga.bash

empty
Jtag frequency : requested 6.00MHz -> real 6.00MHz
Parse file Parse impl/pnr/fpga_project.fs:
Done
DONE
Load SRAM: [=====] 100.00%
Done
DONE
CRC check: Success

```

Figura 12: Ejecución exitosa.

5.3. Otros Scripts Útiles

Script	Descripción
03_synthesize_for_fpga.bash	Sintetiza el diseño y programa la FPGA (script principal).
04_configure_fpga.bash	Carga el bitstream previamente compilado sin re-sintetizar.
05_run_gui_for_fpga_synthesis.bash	Abre el entorno gráfico de Gowin IDE.
01_clean.bash	Elimina los archivos temporales generados.

Tabla 1: Scripts disponibles en cada práctica.

5.4. Resumen del Flujo de Trabajo Diario

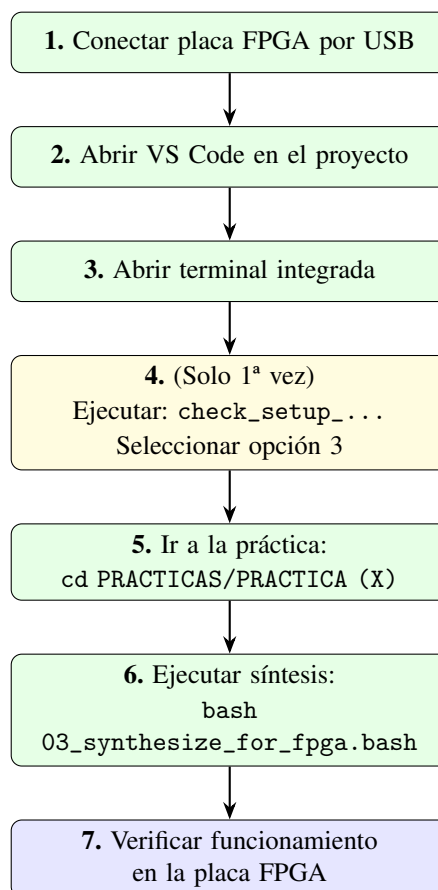


Figura 13: Flujo de trabajo diario para las prácticas (7 pasos).

6. Solución de Problemas Comunes

Error	Solución
<code>gw_sh: command not found</code>	Gowin IDE no está instalado o no está en la ruta esperada.
FPGA not detected	Verificar cable USB, reglas udev (Paso 1), y reconectar la placa.
Multiple drivers	Una misma señal está siendo asignada en más de un lugar en el código.
Error de compilación	Revisar el log: <code>cat Run/*.log grep -i error</code>

Tabla 2: Problemas comunes y sus soluciones.

7. Prácticas de Laboratorio

7.1. Práctica 1: Encendido de un LED Estático

Objetivo: Familiarizarse con la sintaxis de SystemVerilog y el entorno Gowin realizando una conexión física directa e incondicional.

Metas: Encender el LED indicador 0 de la tarjeta y mantener los otros 7 LEDs apagados de manera permanente.

Concepto Clave (Asignación Permanente - assign): En diseño de hardware, una instrucción assign representa una soldadura o cableado permanente. La señal del lado derecho se conecta de forma continua al lado izquierdo del operador.

```
1 // Solo se muestra la asignación del LED; las señales no usadas se  
  ponen en 0.  
2 assign led = 8'b00000001; // bit0 = 1, todos los demás = 0  
3
```

Listing 1: Fragmento esencial de la Práctica 1

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA1/PRACTICA1_top.sv

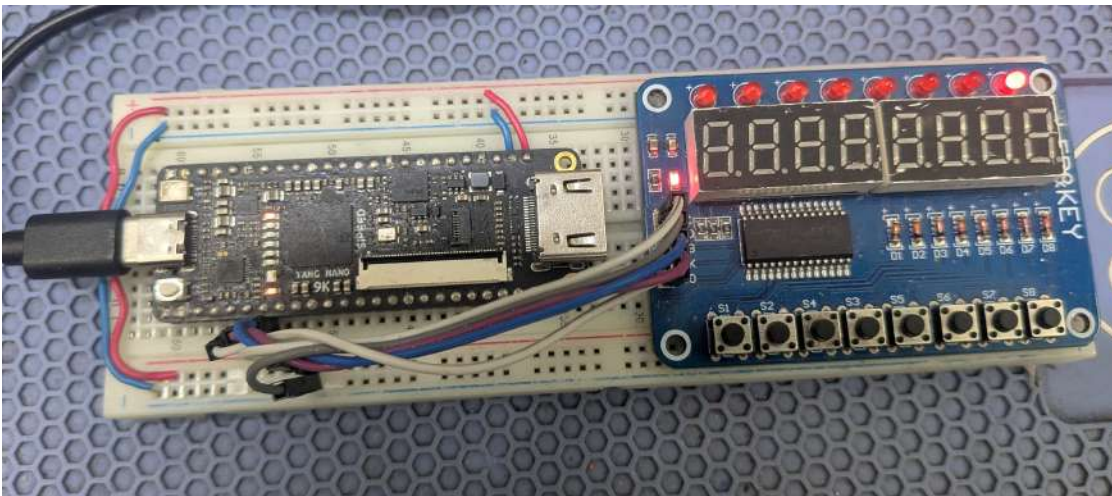


Figura 14: Resultado del encendido del LED físico.

7.2. Práctica 2: El Botón que enciende el LED

Objetivo: Leer entradas físicas en tiempo real y conectarlas a salidas lógicas del FPGA.

Metas: Controlar el LED 0 mediante la pulsación del Botón 0.

Concepto Clave (Lógica Negativa y Concatenación): Muchos botones de desarrollo se conectan con resistencias de *pull-up*, lo que significa que el pin lee un estado alto (1'b1) cuando no se está presionando, y un estado bajo (1'b0) cuando se presiona (conexión a tierra). Para que la salida responda intuitivamente (Botón presionado = LED encendido), se debe invertir la señal con el operador NOT (~).

```
1 // Asignamos el bus de LEDs usando concatenación:
2 assign led = {7'b0000000, ~key[0]};
3 // Los 7 bits superiores a 0, el bit 0 sigue al botón invertido.
4
```

Listing 2: Fragmento esencial de la Práctica 2

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA2/PRACTICA2_top.sv

7.3. Práctica 3: Compuertas Lógicas Básicas

Objetivo: Implementar funciones lógicas booleanas combinacionales elementales.

Metas: Evaluar compuertas AND, OR y XOR usando dos botones y tres LEDs como salidas.

Concepto Clave (Operadores de Bit vs Operadores Lógicos): En HDLs, los operadores a nivel de bits (&, |, ^, ~) actúan individualmente en cada bit físico de la señal. Los operadores lógicos (&&, ||, !) devuelven un valor booleano condicional (verdadero/falso) para control de flujo. En lógica combinacional, se deben utilizar siempre operadores de bits.

```

1      // Convertir botones a lógica positiva
2      logic b0, b1;
3      assign b0 = ~key[0];
4      assign b1 = ~key[1];
5
6      // Compuertas
7      assign led[0] = b0 & b1;    // AND
8      assign led[1] = b0 | b1;    // OR
9      assign led[2] = b0 ^ b1;    // XOR
10

```

Listing 3: Fragmento esencial de la Práctica 3

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA3/PRACTICA3_top.sv

Puerta	Símbolo	Notación	Tabla de verdad																		
AND		$A \cdot B$	<table><tr><th colspan="3">INPUT OUTPUT</th></tr><tr><th>A</th><th>B</th><th>A AND B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	INPUT OUTPUT			A	B	A AND B	0	0	0	0	1	0	1	0	0	1	1	1
INPUT OUTPUT																					
A	B	A AND B																			
0	0	0																			
0	1	0																			
1	0	0																			
1	1	1																			
OR		$A + B$	<table><tr><th colspan="3">INPUT OUTPUT</th></tr><tr><th>A</th><th>B</th><th>A OR B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	INPUT OUTPUT			A	B	A OR B	0	0	0	0	1	1	1	0	1	1	1	1
INPUT OUTPUT																					
A	B	A OR B																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	1																			
NOT		$\bar{A}$	<table><tr><th colspan="2">INPUT OUTPUT</th></tr><tr><th>A</th><th>NOT A</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	INPUT OUTPUT		A	NOT A	0	1	1	0										
INPUT OUTPUT																					
A	NOT A																				
0	1																				
1	0																				
NAND		$\overline{A \cdot B}$	<table><tr><th colspan="3">INPUT OUTPUT</th></tr><tr><th>A</th><th>B</th><th>A NAND B</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	INPUT OUTPUT			A	B	A NAND B	0	0	1	0	1	1	1	0	1	1	1	0
INPUT OUTPUT																					
A	B	A NAND B																			
0	0	1																			
0	1	1																			
1	0	1																			
1	1	0																			
NOR		$\overline{A + B}$	<table><tr><th colspan="3">INPUT OUTPUT</th></tr><tr><th>A</th><th>B</th><th>A NOR B</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	INPUT OUTPUT			A	B	A NOR B	0	0	1	0	1	0	1	0	0	1	1	0
INPUT OUTPUT																					
A	B	A NOR B																			
0	0	1																			
0	1	0																			
1	0	0																			
1	1	0																			
XOR		$A \oplus B$	<table><tr><th colspan="3">INPUT OUTPUT</th></tr><tr><th>A</th><th>B</th><th>A XOR B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	INPUT OUTPUT			A	B	A XOR B	0	0	0	0	1	1	1	0	1	1	1	0
INPUT OUTPUT																					
A	B	A XOR B																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	0																			

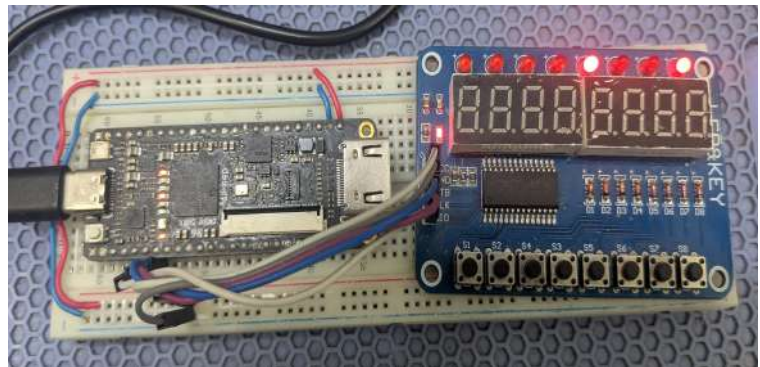


Figura 15: Prueba física de compuertas lógicas.

7.4. Práctica 4: El LED que Parpadea

Objetivo: Introducir lógica secuencial y división de frecuencia de reloj.

Metas: Hacer que un LED parpadee a una frecuencia visible (0.8 Hz) a partir del reloj maestro de 27 MHz de la tarjeta.

Concepto Clave (Lógica Secuencial y División por Contador): Las asignaciones secuenciales se ejecutan con el flanco de subida de un reloj (posedge clock) dentro de un bloque always_ff. Al sumar 1 en cada ciclo de reloj a un contador binario de 25 bits, cada bit actúa como un divisor de frecuencia por potencias de 2:

$$\text{Frecuencia del bit } N = \frac{f_{clk}}{2^{N+1}}$$

Para el bit 24 (MSB): $\frac{27\text{MHz}}{2^{25}} \approx 0.8\text{Hz}$ (un periodo completo de 1.24 segundos).

```
1  logic [24:0] counter;
2  always_ff @(posedge clock or posedge reset) begin
3  if (reset) counter <= 25'b0;
4  else      counter <= counter + 25'b1;
5  end
6  assign led[0] = counter[24]; // el bit más significativo parpadea
7
```

Listing 4: Fragmento esencial de la Práctica 4

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA4/PRACTICA4_top.sv

7.5. Práctica 5: Contador Binario de 8 Bits

Objetivo: Observar la representación de datos binarios y el incremento secuencial rápido.

Metas: Mostrar el conteo binario continuo de 0 a 255 en los 8 LEDs usando un reloj lento.

Concepto Clave (Reloj Lento - slow_clock): Un contador rápido de 27 MHz es imposible de observar a simple vista si se conecta directamente a los LEDs. La placa provee una señal de reloj lento (slow_clock) de aproximadamente 100 Hz, la cual permite apreciar perfectamente el avance de bits en los LEDs.

```
1  logic [7:0] counter;  
2  always_ff @(posedge slow_clock or posedge reset) begin  
3  if (reset) counter <= 8'h00;  
4  else      counter <= counter + 8'h01;  
5  end  
6  assign led = counter;  
7
```

Listing 5: Fragmento esencial de la Práctica 5

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA5/PRACTICA5_top.sv

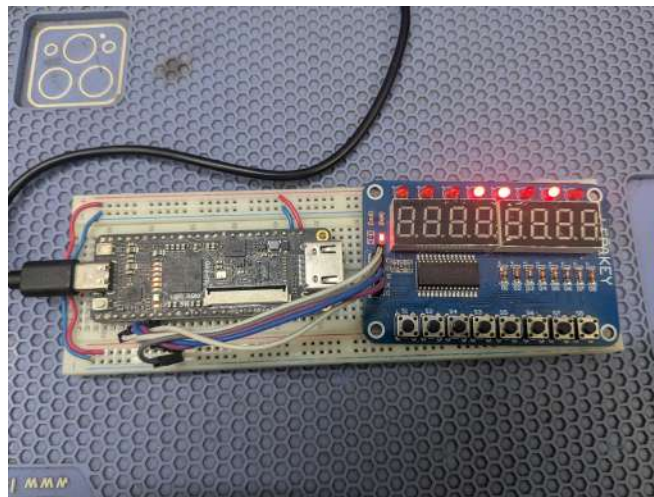


Figura 16: Conteo en binario de 8 bits.

7.6. Práctica 6: Display de 7 Segmentos (Dígito Simple)

Objetivo: Aprender a decodificar valores numéricos binarios en combinaciones de LEDs para lectura humana.

Metas: Crear un contador decimal de 0 a 9 y mostrarlo en el primer dígito del display de 7 segmentos.

Concepto Clave (Mapeo de Bits para 7 Segmentos): El display de la tarjeta usa lógica de cátodo común (1 = segmento encendido). El orden de bits del bus abcdefgh entregado a la tarjeta es de derecha a izquierda: {h, g, f, e, d, c, b, a}, donde a es el LSB (segmento superior) y h es el MSB (punto decimal).

```
1 function automatic [7:0] seg7_decode(input [3:0] num);
2 case (num)
3 4'h0: seg7_decode = 8'b00111111; // 0
4 4'h1: seg7_decode = 8'b00000110; // 1
5 // ... resto de dígitos
6 default: seg7_decode = 8'b00000000;
7 endcase
8 endfunction
9
```

Listing 6: Fragmento esencial de la Práctica 6 (decodificador)

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA6/PRACTICA6_top.sv

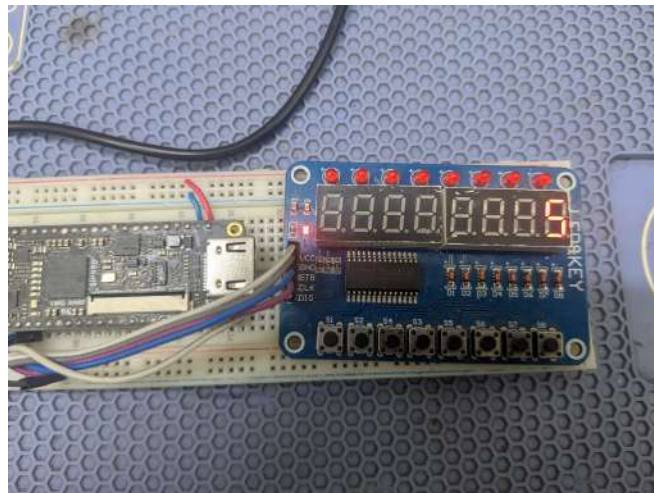


Figura 17: Visualización en 7 segmentos.

7.7. Práctica 7: Multiplexación de Display y Antirrebote

Objetivo: Mostrar números de múltiples dígitos y filtrar los rebotes mecánicos de los botones.

Metas: Incrementar un contador decimal de 0 a 255 al presionar el Botón 0 y mostrarlo en tres dígitos multiplexados del display de 7 segmentos.

Concepto Clave (Multiplexación y Antirrebote - Debouncing):

- **Antirrebote:** Los botones físicos vibran mecánicamente al presionarse. Un circuito digital debe registrar el cambio solo cuando la señal de entrada se mantiene estable por varios milisegundos.
- **Multiplexación:** Para no duplicar cables, se envían los datos de cada dígito en secuencia rápida (500 Hz). El ojo humano percibe todos los dígitos encendidos a la vez gracias a la persistencia de la visión.

```
1 // Detección de flanco estable
2 wire b0_edge = b0_stable & ~b0_prev; // flanco de subida limpio
3 // Incrementar contador solo en ese flanco
4 always_ff @(posedge slow_clock or posedge reset) begin
5   if (reset) counter <= 0;
6   else if (b0_edge) counter <= counter + 1;
7   end
8
```

Listing 7: Fragmento del antirrebote (Práctica 7)

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA7/PRACTICA7_top.sv

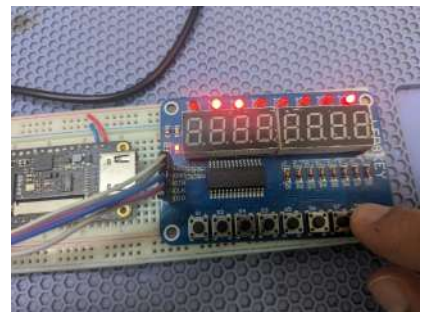


Figura 18: Multiplexación de display de 3 dígitos.

7.8. Práctica 8: Semáforo con LEDs (Máquina de Estados)

Objetivo: Diseñar sistemas de control secuencial estructurados.

Metas: Controlar tres LEDs (rojo, amarillo, verde) emulando la secuencia cíclica y cronometrada de un semáforo de tránsito real.

Concepto Clave (FSM - Máquina de Estados Finitos): Las máquinas de estado dividen el comportamiento del circuito en estados lógicos definidos. Usar tipos de datos enumerados (`typedef enum`) evita el uso de valores numéricos directos o "mágicos", facilitando la lectura del código y permitiendo al compilador realizar optimizaciones seguras.

```
1      typedef enum logic [1:0] { ROJO, ROJO_AMARILLO, VERDE, AMARILLO }  
estado_t;  
2      estado_t estado;  
3      always_ff @(posedge slow_clock) begin  
4          case (estado)  
5              ROJO:          if (cuenta == 0) estado <= ROJO_AMARILLO;  
6              ROJO_AMARILLO: if (cuenta == 0) estado <= VERDE;  
7              // ... transiciones  
8          endcase  
9      end  
10
```

Listing 8: Fragmento de la FSM (Práctica 8)

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA8/PRACTICA8_top.sv

7.9. Práctica 9: Introducción a Gráficos LCD (Coordenadas y Colores)

Objetivo: Comprender el funcionamiento combinacional de un generador de gráficos en una pantalla digital.

Metas: Dibujar tres rectángulos estáticos de colores diferentes (rojo, verde, azul) en posiciones específicas sobre la pantalla LCD.

Concepto Clave (Resolución LCD y Coordenadas): La pantalla del kit de desarrollo tiene una resolución activa de **480x272 píxeles**. El bloque combinacional recibe constantemente las coordenadas x (0-479) e y (0-271) del píxel que la pantalla está dibujando en ese instante de tiempo, y debe decidir qué color asignarle a la salida en paralelo.

```
1  always_comb begin
2      red = 5'h00; green = 6'h00; blue = 5'h00; // fondo negro
3      if (x > 50 && x < 150 && y > 50 && y < 100) begin
4          red = 5'h1F; green = 6'h00; blue = 5'h00; // rojo
5      end
6      // ... otros rectángulos
7  end
8
```

Listing 9: Fragmento del dibujo combinacional (Práctica 9)

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA9/PRACTICA9_top.sv

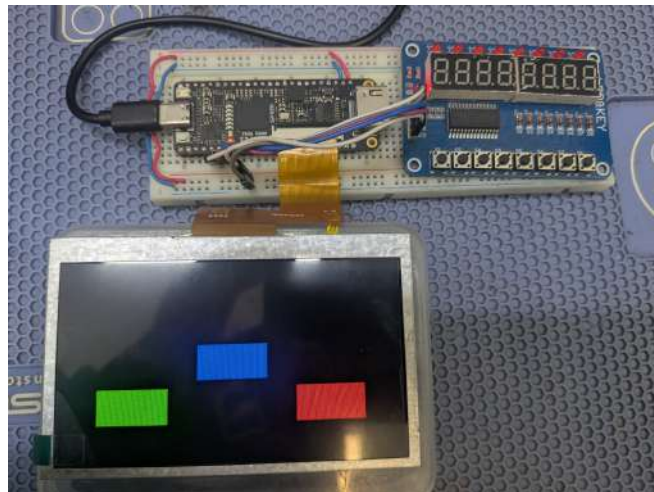


Figura 19: Gráficos estáticos en pantalla LCD.

7.10. Práctica 10: Movimiento Automático en Pantalla LCD

Objetivo: Sincronizar eventos lógicos con el refresco de cuadro de video.

Metas: Crear una animación donde un rectángulo de color amarillo se desplace horizontalmente por la pantalla de izquierda a derecha de forma infinita.

Concepto Clave (Señales de Sincronismo y Cuadros por Segundo): El movimiento requiere actualizar la posición de la figura una única vez por cada cuadro de video (tasa habitual de 30 o 60 Hz). El módulo `strobe_gen` genera un pulso de un ciclo de duración a la frecuencia deseada para actualizar el registro de posición de la figura.

```
1      logic enable;  
2      strobe_gen #(.clk_mhz(27), .strobe_hz(30)) i_strobe (.clk(clock), .  
rst(reset), .strobe(enable));  
3      always_ff @(posedge clock) begin  
4          if (enable) begin  
5              if (pos_x >= 400) pos_x <= 0;  
6              else pos_x <= pos_x + 1;  
7          end  
8      end  
9
```

Listing 10: Fragmento de la animación (Práctica 10)

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA10/PRACTICA10_top.sv



Figura 20: Animación de objeto horizontal.

7.11. Práctica 11: Control Interactivo por Botones

Objetivo: Combinar lecturas de entrada asíncronas con la generación dinámica de gráficos.

Metas: Mover un cuadrado blanco en dos dimensiones (arriba, abajo, izquierda, derecha) utilizando cuatro botones de la placa de desarrollo.

Concepto Clave (Control Bidimensional e Interfaces): Se mapean las entradas asíncronas de los botones a variables locales de dirección. En cada ciclo de reloj lento, si el botón está activo, se modifica la coordenada de la figura, validando límites geométricos estrictos para que el objeto no desaparezca de la pantalla activa.

```
1  logic left, right, up, down;
2  assign left = ~key[0]; assign right = ~key[1];
3  assign up   = ~key[2]; assign down  = ~key[3];
4  always_ff @(posedge slow_clock) begin
5  if (left && pos_x > 5)   pos_x <= pos_x - 5;
6  if (right && pos_x < 425) pos_x <= pos_x + 5;
7  // ... movimiento vertical
8  end
9
```

Listing 11: Fragmento del control (Práctica 11)

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA11/PRACTICA11_top.sv

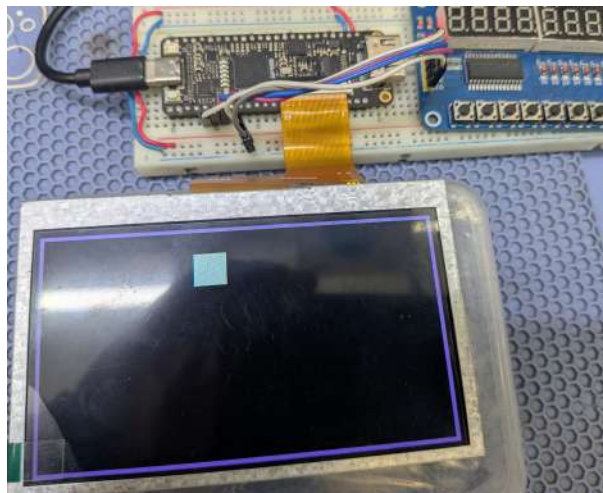


Figura 21: Animación de objeto controlado por botones.

7.12. Práctica 12: Pelota Rebotante (Animación Física LCD)

Objetivo: Implementar algoritmos de simulación física simples en hardware.

Metas: Simular el movimiento continuo y el rebote de una pelota circular roja contra los límites físicos de la pantalla de 480x272 píxeles.

Concepto Clave (Límites de Colisión Simétricos): Para un objeto circular con un radio $R = 20$ píxeles, la pelota colisiona y debe rebotar (invertir su dirección de velocidad dir) cuando el centro geométrico de la pelota toca los límites físicos ajustados: * Límite Izquierdo: $X \leq 20$. Límite Derecho: $X \geq 460$ ($480 - 20$). * Límite Superior: $Y \leq 20$. Límite Inferior: $Y \geq 252$ ($272 - 20$).

```
1  always_ff @(posedge clock) begin
2  if (dir_x == 0) begin
3  if (pos_x >= 460) dir_x <= 1; else pos_x <= pos_x + 4;
4  end else begin
5  if (pos_x <= 20) dir_x <= 0; else pos_x <= pos_x - 4;
6  end
7  // análogo para dir_y
8  end
9
```

Listing 12: Fragmento de la física (Práctica 12)

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA12/PRACTICA12_top.sv

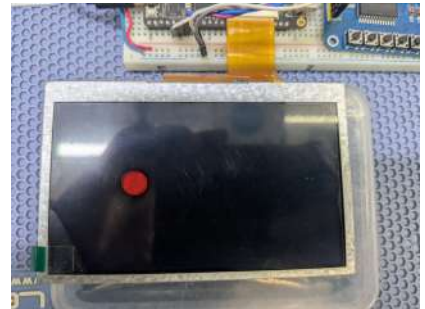
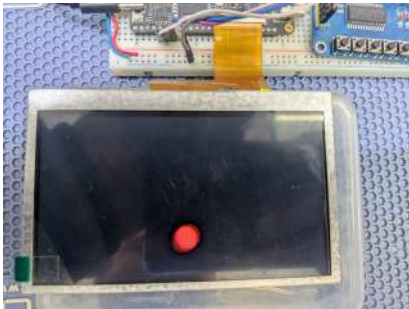


Figura 22: Física de colisión elástica en LCD.

7.13. Práctica 13: Semáforo Animado en LCD

Objetivo: Combinar máquinas de estado lógicas con renderizadores complejos de video.

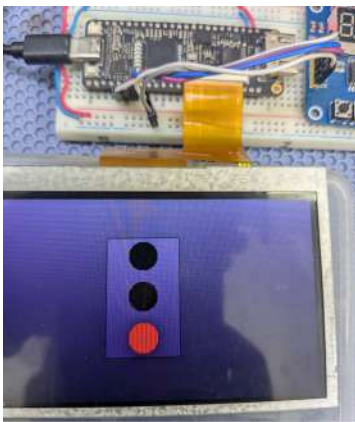
Metas: Dibujar la silueta de un semáforo en pantalla con tres luces circulares (roja, amarilla y verde) que se encienden cíclicamente de forma sincronizada con una máquina de estados internos.

Concepto Clave (Dimensionamiento Geométrico No Superpuesto): Para lograr un acabado gráfico limpio, las dimensiones de los elementos dibujados deben calcularse matemáticamente. Si se define un semáforo centrado en $Y = 136$ con tres lámparas de radio $R = 20$ y un espaciado vertical de centros de 55 píxeles, las luces se renderizarán a $Y_{rojo} = 81$, $Y_{amarillo} = 136$, $Y_{verde} = 191$ sin traslaparse en ningún momento y encajando dentro del marco gris de la caja.

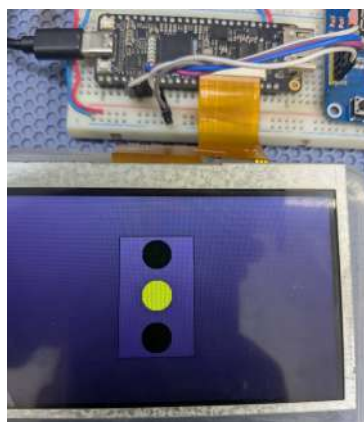
```
1      localparam CENTRO_X = 240, CENTRO_Y = 136, RADIO = 20, SEP = 55;  
2      localparam ROJO_Y = CENTRO_Y - SEP, AMARILLO_Y = CENTRO_Y, VERDE_Y  
3      = CENTRO_Y + SEP;  
4      always_comb begin  
5          // dibujo del cuerpo y las luces condicionadas al estado FSM  
6          end
```

Listing 13: Fragmento del semáforo LCD (Práctica 13)

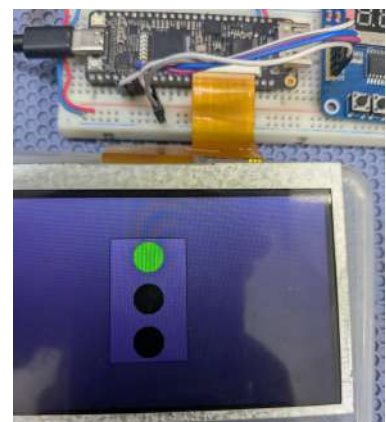
Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA13/PRACTICA13_top.sv



Luz roja



Luz amarilla



Luz verde

Figura 23: Semáforo gráfico sincronizado con FSM: fases roja, amarilla y verde.

7.14. Práctica 14: Radar y Medición de Distancia Ultrasónica

Objetivo: Realizar la integración de periféricos de terceros, conversión matemática de escala y control con alarmas visuales.

Metas: Leer la distancia de un sensor HC-SR04, mostrar la medición en centímetros reales en el display de 7 segmentos y encender el LED 0 de alarma si un objeto se acerca a menos de 30 cm.

Concepto Clave (Resolución de Conflictos de Señales y Conversión Física):

- **Múltiple Driver:** Si un pin físico se declara con un assign directo a cero y a la vez se le asigna la salida de un módulo secundario, el compilador arrojará un error. Se debe asegurar un único origen de excitación para cada puerto.
- **Conversión Física de Ticks a Centímetros:** El sonido viaja a 343 m/s. A una frecuencia de 27 MHz, el sensor cuenta el número de ciclos (echo_cnt). Dado que la señal va y viene:

$$\text{Ciclos por Centímetro} = \frac{27 \times 10^6 \text{ Hz} \times 2}{343 \times 100} \approx 1574 \text{ ciclos/cm}$$

Dado que el submódulo del sensor entrega una medición escalada de $\text{distance} = \text{echo_cnt} / 16$, los centímetros reales son:

$$\text{Distancia (cm)} = \frac{\text{distance} \times 16}{1574} \approx \frac{\text{distance}}{98}$$

```

1      wire [15:0] distance;
2      ultrasonic_distance_sensor #(.clk_frequency(27_000_000)) i_sensor
3      (...);
4      assign distance_cm = distance / 16'd98;
5      always_comb begin
6          led = 0;
7          if (distance_cm < 30) led[0] = 1'b1; // alarma
8          end

```

Listing 14: Fragmento de la medición (Práctica 14)

Repositorio GitHub: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA14/PRACTICA14_top.sv

8. Extendiendo el Proyecto: Agregando Periféricos

Una vez completadas las prácticas básicas, el sistema puede ampliarse para incluir otros periféricos o protocolos de comunicación. A continuación se dan lineamientos generales.

8.1. Agregar un Módulo Personalizado

Cada periférico debe encapsularse en un módulo SystemVerilog independiente. Por ejemplo, para añadir un sensor de temperatura I2C:

1. Escriba el módulo (e.g., `i2c_master.sv`) respetando las señales de reloj y reset del sistema.
2. Inserte la instancia en el módulo top:

```
1         i2c_master #(.CLK_FREQ(27_000_000)) i_i2c (  
2             .clk(clock), .rst(reset), .sda(gpio[2]), .scl(gpio[3]), .  
           data(temp_data)  
3             );  
4
```

3. Conecte las señales de datos a la lógica de visualización (display LCD o 7 segmentos).
4. Modifique el archivo de restricciones físicas (`.cst`) para asignar los pines GPIO correspondientes.

8.2. Mapeo de Pines

Revise el esquemático de la Tang Nano 9K y el módulo de expansión para seleccionar pines disponibles. La asignación de pines se realiza en el **Gowin IDE** a través del editor de restricciones o editando directamente el archivo `.cst`. Ejemplo:

```
IO_LOC "gpio[0]" 10, 3, 1; // pin físico  
IO_PORT "gpio[0]" PULL_MODE=UP;
```

8.3. Síntesis Automatizada

Actualice el script `03_synthesize_for_fpga.bash` (o el archivo TCL) para incluir los nuevos fuentes. De esta forma, al ejecutar el script se sintetizará el proyecto completo de manera consistente. Los repositorios de GitHub incluyen ejemplos de estos scripts.

8.4. Recomendaciones

- Pruebe cada periférico por separado antes de integrarlo en el sistema completo.
- Utilice simulaciones (por ejemplo, con ModelSim o GHDL) para verificar la lógica antes de cargarla en la FPGA.
- Documente en el repositorio de GitHub la descripción del periférico y los cambios realizados.

9. Recomendaciones de Diseño y Buenas Prácticas

Al diseñar sistemas digitales sobre FPGA en SystemVerilog, es fundamental seguir reglas estrictas para asegurar que el circuito sintetizado sea confiable en hardware:

1. **Evitar Múltiple Driver:** Una señal sólo puede ser asignada mediante un bloque `always` o una sentencia `assign`. Conectar múltiples fuentes a una señal generará conflictos de contención eléctrica y errores de compilación críticos.
2. **Diseño Totalmente Síncrono:** Utilizar siempre el reloj maestro (`clock`) como señal de reloj en los bloques secuenciales `always_ff`. Para velocidades menores, usar habilitaciones de reloj (`clock enables`) en lugar de crear relojes divididos por lógica combinacional, lo que incrementa el desfase de reloj (`*clock skew*`).
3. **Asignación de Señales:**
 - Usar asignación no bloqueante (`<=`) dentro de bloques secuenciales `always_ff`.
 - Usar asignación bloqueante (`=`) dentro de bloques combinacionales `always_comb`.
 - Nunca mezclar ambos tipos de asignación en el mismo bloque.
4. **Declaración de Buses:** Utilizar el tipo `logic` de SystemVerilog en lugar de los tipos tradicionales de Verilog `wire` y `reg`. El tipo `logic` previene de manera automática dobles asignaciones accidentales durante el análisis de síntesis.

10. Conclusiones

Este manual de prácticas sirve como un andamiaje didáctico progresivo para que cualquier estudiante, comenzando desde cero, pueda asimilar conceptos teóricos de electrónica digital y llevarlos directamente a la ejecución física sobre el FPGA Tang Nano 9K. El aprendizaje incremental, desde el encendido de un simple LED hasta el procesamiento y escalado digital de señales analógicas de sensores ultrasónicos, dota al estudiante de las capacidades prácticas indispensables en el diseño de hardware actual.

11. Anexo de Ecuaciones

Ecuación de la velocidad del sonido para el sensor ultrasónico:

$$d = \frac{v \times t}{2}$$

Donde: * d : Distancia en metros. * v : Velocidad del sonido en el aire (≈ 343 m/s). * t : Tiempo transcurrido del eco en segundos.

Relación de división de frecuencia por contador binario:

$$f_{\text{out}} = \frac{f_{\text{in}}}{2^{N+1}}$$